When an `AuditorAware` bean is defined and auditing is enabled, Spring Data will use it to extract the principal. In our case, it's the username of the currently authenticated user, represented as a `String`. Then we can use `@CreatedBy` and `@LastModifiedBy` to annotate two new fields in the `Book` record. They will be populated by Spring Data automatically whenever a create or update operation is performed on the entity.

Listing 12.22 Fields to capture user audit metadata in a JDBC entity

```
public record Book (
  ...

  @CreatedBy                    ❶
  String createdBy,

  @LastModifiedBy               ❷
  String lastModifiedBy,

){
  public static Book of(String isbn, String title, String author,
    Double price, String publisher
  ) {
    return new Book(null, isbn, title, author, price, publisher,
      null, null, null, null, 0);
  }
}
```

❶ Who created the entity

❷ Who modified the entity the last time

After adding the new fields, we need to update a few classes using the `Book` all-args constructor, which now requires passing values for `createdBy` and `lastModifiedBy`.

The `BookService` class contains the logic for updating books. Open it and change the `editBookDetails()` method to ensure the audit metadata is correctly passed along when calling the data layer.